

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	AKSHAYA KUMAR CHANDRASEKAR	Reg. No.:	192425350
Section / Batch:	B.TECH AIML	Date:	28-07-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Debugging Pair Sum Code (Output Variables Not Validated in C)

The following brute force Pair Sum implementation in C assumes that the output pointers a and b are always valid, which can cause runtime errors if null pointers are passed by the caller. Carefully analyze why pointer validation is important in C before writing results through indirection. Apply systematic debugging so that the function safely checks its output parameters and reports failure appropriately when they are invalid. Optimize the corrected implementation for defensive programming and clear API behavior. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
```

```

*a = i; /* ISSUE if a == NULL */
*b = j; /* ISSUE if b == NULL */
return 1;
}
}
}
return 0;
}

```

main.py

Share

Run

Output

```

1- def pair_sum(arr, target):
2-     # Validate input
3-     if arr is None:
4-         return None
5-
6-     # Check all possible pairs
7-     for i in range(len(arr)):
8-         for j in range(i + 1, len(arr)):
9-             if arr[i] + arr[j] == target:
10-                 return (i, j) # Return indices
11-
12-     return None # Pair not found
13-
14-
15- # Main Program
16- arr = [2, 7, 11, 15]
17- target = 9
18-
19- result = pair_sum(arr, target)
20-
21- if result is not None:
22-     print("Pair found at indices:", result[0], "and", result[1])

```

Pair found at indices: 0 and 1

=== Code Execution Successful ===

Code of Conduct

I AKSHAYA KUMAR CHANDRASEKAR(192425350)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Akshaya kumar

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues

Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____